```
import java.util.concurrent.atomic.AtomicLong
public class SabbirThread extends Thread{
  static AtomicLong safeCount = new AtomicLong();
  static long unsafeCount = 0;
  static boolean safe;
  static int n;
  long nonStaticCount = 0;
  SabbirThread (int n) {
      SabbirThread.n = n;
  }
  public void run (){
     for( int i=0; i<n; i++){
       if (safe)
           safeCount.incrementAndGet();
       else
           unsafeCount++;
       nonstaticCount++;
     }
  }
```

```
public static void main (String[] args)
 throw. InterruptedException {
  int thread = Integer.parseInt (args[0]);
  n = Integer. parseInt (args [1]);
  safe = Boolean.parseBoolean (args[2]);
  SabbirThread [] t = new SabbirThread [threads];
  for (int i=0 i <threads; i++) {
    t[i] = new SabbirThread (n);
    t[i].start ();
  }
  long total = 0;
  for (int i=0; i<threads; i++) {
    t[i].join ();
    total += t[i]. nonStaticCount;
}-
long staticCount = safe
      ? safeCount. get (): unsafeCount;
```

```
long expected = (long) threads * n;
long difference = Math.abs(staticCount - total);

double percentage = total == 0
    ? (staticCount == 0 ? 0 : Double.NaN)
    : difference * 100.0 / total;

System.out.println("Expected Count = " + expected);
System.out.println("Static Count = " + staticCount);
System.out.println("Non-static Count = " + total);
System.out.println("Absolute Difference = " + diff);
System.out.println("Percentage Difference
        = %.2f%%%n", percentage);
}
}
```